Week 5 Assignment Reflection

Intelligent Anomaly Detection and AI-Generated Root Cause Analysis

WHAT I BUILT

A small pipeline: `generate_data.py` makes labelled synthetic metrics and matching structured logs. `anomaly_detector.py` runs Isolation Forest (and DBSCAN for comparison) and checks precision/recall/F1 against the known ground truth. `alert_grouper.py` turns the 16 anomalous rows into 48 per-metric alerts and groups them by time gap into one incident. `rca_agent.py` runs an agentic RCA on that incident using two tools (`get_metrics`, `get_logs`) plus a synthesis step, all instrumented with OTel spans following the GenAI conventions.

KEY DECISIONS

I picked Isolation Forest as the main detector because `contamination` is an easy parameter to reason about for the expected anomaly rate. I used rule-based (time-gap) grouping instead of an LLM for alerts because it is a simple, deterministic problem that does not need an LLM call. Using an agent where a five-line rule works fine would just waste money and time.

WHAT DIDN'T WORK AS EXPECTED, HONESTLY

- `uv` was not installed on my machine at all. I had to `pip install --user uv`, and even then the installed `uv.exe` was not on `PATH`, so I ended up running `python -m uv` instead of a plain `uv` command the whole time.
- `uv init` sets up a packaged project by default (a package folder plus a script entry point and a build backend). That is the wrong shape for a folder of plain scripts, so I had to delete the generated package and add `[tool.uv] package = false` before `uv sync` would stop trying to build the project as a wheel.
- The notebook cells that call `log_analyzer_agent.py` and `rca_agent.py` through `subprocess.run(["python", ...])` failed the first time with `ModuleNotFoundError: No module named 'opentelemetry'`, even though the notebook itself was clearly running inside the project's venv. Turned out a plain `"python"` string just grabs whatever `python` is first on `PATH`, which was not the venv's interpreter. Switching to `sys.executable` fixed it right away. I had not realized a subprocess launched from a Jupyter kernel does not automatically use the same interpreter.
- DBSCAN did much worse than Isolation Forest (F1 about 0.48 vs. 0.89) with `eps=1.0, min_samples=5`. This one actually surprised me. I expected DBSCAN to do fine since the incident is basically one dense cluster of 16 points, which sounds like exactly what DBSCAN should find. The problem is that a dense cluster of anomalies looks the same to DBSCAN as a dense cluster of normal points. Density alone does not tell you "this is far from normal," just "this is close to its neighbors." Isolation Forest looks at how typical a point is compared to the whole dataset, which fits this problem better.

WHAT I'D DO DIFFERENTLY WITH MORE TIME

I would actually tune DBSCAN's `eps` and `min_samples` properly instead of guessing one value, maybe using a k-distance plot, to see how close it can get to Isolation Forest. I would also connect the RCA agent's tools to a real Claude call behind a flag, so switching from simulated to live would just be a config change instead of a rewrite. (No `ANTHROPIC_API_KEY` was set up in this environment, so all "LLM calls" in this submission are simulated but based on real detector/log data, see `README.md` for details.)